

Valheim Donations

Complete Implementation & Setup Guide

Backend 0.5.0 · Plugin 5.0.0

Generated 2026-05-30 18:54 UTC · fingerprint d334d3ae6a84

This guide walks you end-to-end through deploying the donation backend, building the BepInEx plugin, hooking up payment providers, and verifying the full donation flow inside Valheim. It is generated directly from the project's READMEs, environment templates, and source files — never hand-edited.

Contents

1. Architecture at a glance
2. Prerequisites
3. Backend — local development
4. Environment variables reference
5. Provider setup (Ko-fi, PayPal, Patreon, PayMongo)
6. Deploying the backend to Fly.io
7. Building the BepInEx plugin
8. Installing the plugin on the server
9. Configuration files & persistent state
10. Chat commands & shop catalog
11. In-game UI panel (Phase 5)
12. Verifying the full flow
13. Operations & troubleshooting
14. Keeping this guide up to date

1. Architecture at a glance

The system is split in two halves that share a single source of truth: the backend's SQLite ledger.

- Backend (FastAPI, Python 3.12+) receives webhooks from each provider, owns the coin ledger, and exposes a polling API.
- Plugin (BepInEx, .NET Framework 4.7.2) runs on the Valheim dedicated server, polls grants every ~10 s, applies them, and drives the chat commands plus the optional client-side panel.
- Donor portal (Jinja templates served by the backend) is what donors see when they paste their claim code — one page, four provider buttons.

Donation flow:

```
/donate in-game
-> plugin POST /api/claim
-> backend mints AB12-CD34 (TTL 30 min)
-> plugin DMs donor: 'Donate at /portal, code is AB12-CD34'
-> donor visits /portal/AB12-CD34
    Ko-fi / PayPal -> hosted page (code prefilled where possible)
    PayMongo       -> portal mints PaymentLink with metadata.claim_code
    Patreon        -> patreon.com + 'Link my account' (OAuth)
-> provider webhook fires -> backend resolves code -> grants row
-> plugin polls /api/grants/pending -> applies -> /api/grants/ack
```

2. Prerequisites

- Python 3.12+ with pip and venv.
- .NET SDK 6.0+ (for `dotnet build -c Release`).
- A Valheim dedicated server with BepInEx already installed.
- Fly.io account + `flyctl` CLI (for the recommended deploy).
- Accounts on whichever providers you want to accept: Ko-fi, PayPal (developer app), Patreon (creator + API client), and/or PayMongo (for GCash/Maya/cards in PH).
- A copy of Valheim's `valheim\_Data/Managed/` folder accessible to your build machine — you need to copy DLLs from it.

3. Backend — local development

Use a fresh virtual environment so you don't pollute your global Python install. PowerShell commands shown; bash works the same way.

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
copy .env.example .env          # then edit secrets
uvicorn app.main:app --reload --port 8080
```

Run the test suite:

```
.\.venv\Scripts\python.exe -m pytest
```

FastAPI app version: **0.5.0**. Routers registered at startup: health, claim, grants, admin, spend, leaderboard, state, portal, kofi, paypal, patreon, paymongo.

4. Environment variables reference

These are extracted from `backend/.env.example`. Each provider's routes return 503 if its config is missing, so partial setups are perfectly fine while you're rolling out one provider at a time.

Variable	Default / example	Notes
PLUGIN_TOKEN	replace-me-with-a-long-random-string	Required
PUBLIC_BASE_URL	https://your-app.fly.dev	—
DONATION_URL	https://your-app.fly.dev/portal	—
KOFI_VERIFICATION_TOKEN	(empty)	--- Ko-fi ---
KOFI_USERNAME	(empty)	for portal deep-links (ko-fi.com/<username>)
PAYPAL_CLIENT_ID	(empty)	--- PayPal ---
PAYPAL_CLIENT_SECRET	(empty)	—
PAYPAL_WEBHOOK_ID	(empty)	—
PAYPAL_SANDBOX	false	—
PAYPAL_ME_USERNAME	(empty)	for portal deep-links (paypal.me/<username>)
PATREON_WEBHOOK_SECRET	(empty)	--- Patreon ---
PATREON_USERNAME	(empty)	patreon.com/<username>
PATREON_CLIENT_ID	(empty)	OAuth (used by the portal's "Link my Patreon account" button):
PATREON_CLIENT_SECRET	(empty)	—
PATREON_REDIRECT_URI	https://your-app.fly.dev/portal/patreon/callback	—
PAYMONGO_WEBHOOK_SECRET	(empty)	--- PayMongo ---
PAYMONGO_SECRET_KEY	(empty)	sk_test_... or sk_live_... — needed to mint PaymentLinks

5. Provider setup

Ko-fi

- Dashboard -> More -> API. Set Webhook URL to `https://<your-app>.fly.dev/webhooks/kofi`.
- Copy the verification token into `KOFI_VERIFICATION_TOKEN`.
- Set `KOFI_USERNAME` so the portal can deep-link with the claim code prefilled as the donation message.

PayPal

- `developer.paypal.com` -> Apps & Credentials -> create REST app.
- On that app: Webhooks -> URL `/webhooks/paypal`. Subscribe to `PAYMENT.CAPTURE.COMPLETED` (and `PAYMENT.SALE.COMPLETED` if you accept classic donations).
- Copy Client ID, Secret, Webhook ID into env. Use `PAYPAL_SANDBOX=true` while testing.
- Set `PAYPAL_ME_USERNAME` for the portal's paypal.me link. Donors paste the claim code into PayPal's note field manually — paypal.me URLs cannot prefill notes. The portal renders the code prominently with a Copy button.

Patreon

- Creator portal -> Settings -> Webhooks -> add `/webhooks/patreon`. Subscribe to `members:pledge:*` plus `members:create` and `members:update`.
- Copy the webhook secret into `PATREON_WEBHOOK_SECRET`.
- For OAuth linking: create an API client in the developer portal with redirect URI `/portal/patreon/callback`. Fill in `PATREON_CLIENT_ID`, `PATREON_CLIENT_SECRET`, `PATREON_REDIRECT_URI`.
- Set `PATREON_USERNAME` for the portal's Patreon link.
- First-time patrons use the 'Link my Patreon account' button to bind their account. Later donations from the same patron are auto-credited via the `provider_links` table.

PayMongo (GCash + Maya + cards)

- `dashboard.paymongo.com` -> Developers -> API Keys. Copy the secret key into `PAYMONGO_SECRET_KEY`. This is required to mint PaymentLinks.
- Developers -> Webhooks -> add `/webhooks/paymongo` with the event `payment.paid`.
- Copy the signing secret into `PAYMONGO_WEBHOOK_SECRET`.
- The portal mints a PaymentLink with `metadata.claim_code` baked in when the donor clicks Pay — no manual code entry needed.

6. Deploying the backend to Fly.io

Defaults baked into `fly.toml`: region **sin**, shared CPU, **256 MB** RAM, 1 GB persistent volume mounted at `/data` for SQLite.

```
flyctl auth login
flyctl launch --no-deploy          # claims app name, rewrites fly.toml
flyctl volumes create valcoin_data --size 1 --region sin
flyctl secrets set \
  PLUGIN_TOKEN="$(python -c 'import secrets;print(secrets.token_urlsafe(32))')" \
  PUBLIC_BASE_URL="https://<your-app>.fly.dev" \
  DONATION_URL="https://<your-app>.fly.dev/portal" \
  KOFI_VERIFICATION_TOKEN="..." KOFI_USERNAME="yourname"
flyctl deploy
```

Per-provider secrets (set after their accounts are wired up):

```
flyctl secrets set \
  PAYPAL_CLIENT_ID="..." PAYPAL_CLIENT_SECRET="..." \
  PAYPAL_WEBHOOK_ID="..." PAYPAL_ME_USERNAME="..." \
  PATREON_WEBHOOK_SECRET="..." PATREON_USERNAME="..." \
  PATREON_CLIENT_ID="..." PATREON_CLIENT_SECRET="..." \
  PATREON_REDIRECT_URI="https://<your-app>.fly.dev/portal/patreon/callback" \
  PAYMONGO_WEBHOOK_SECRET="..." PAYMONGO_SECRET_KEY="sk_live..."
```

Tip: The app name in `fly.toml` ships as the placeholder `valheim-donations`. `flyctl launch --no-deploy` rewrites it to whatever name you pick.

7. Building the BepInEx plugin

The csproj expects the following DLLs in `valheim-plugin/libs/`. Anything starred is not in source control — copy it from your local Valheim install at `valheim_Data/Managed/`.

DLL	Status
0Harmony.dll	present in libs/
UnityEngine.dll	present in libs/
UnityEngine.CoreModule.dll	present in libs/
UnityEngine.UnityWebRequestModule.dll	MISSING — copy from valheim_Data/Managed/
UnityEngine.IMGUIModule.dll	MISSING — copy from valheim_Data/Managed/
UnityEngine.InputLegacyModule.dll	MISSING — copy from valheim_Data/Managed/
assembly_valheim.dll	present in libs/
BepInEx.dll	present in libs/
Newtonsoft.Json.dll	present in libs/

```
cd valheim-plugin
dotnet build -c Release
# Output: bin\Release\net472\ValheimDonationSystem.dll
```

Plugin version: 5.0.0. The existing bin/Release/net472/ output bundles Jotunn.dll and YamlDotNet.dll, but the current csproj no longer references either — that artifact predates the dependency cleanup. Always rebuild before shipping.

8. Installing the plugin on the server

- Copy `ValheimDonationSystem.dll` into the server's `BepInEx/plugins/` folder.
- Start the server once so the plugin generates template config files in `BepInEx/config/`.
- Edit `valcoin_config.json`: set `backend_url` to your Fly app URL and `plugin_token` to match the `PLUGIN_TOKEN` secret on the backend.
- Edit `valcoin_admins.yaml`: replace the placeholder `Steam64` with your own admin IDs.
- Restart the server. Watch the log for '[Valheim Donations] Plugin loaded' and 'Startup complete. Admins: N, Backend ready: True'.

Optional: install client-side too

If players install the same DLL into their own `BepInEx/plugins/`, they get the F8 in-game panel. Vanilla clients keep working via chat commands — nothing is required of them.

Environment overrides: `VALCOIN_BACKEND_URL` and `VALCOIN_PLUGIN_TOKEN` beat the JSON file. Useful on hosts that expose env vars but not config files.

9. Configuration files & persistent state

Path	Purpose
BepInEx/config/valcoin_config.json	Backend URL + bearer token + poll interval
BepInEx/config/valcoin_admins.yaml	Admin Steam64 ID allowlist
BepInEx/config/valcoin_shop.yaml	SKU catalog (auto-generated)
BepInEx/config/valcoin_data/coin_balances.json	Local balance cache + applied-grant dedupe
BepInEx/config/valcoin_data/perks.json	Per-player perks, charges, title, /home
/data/valcoin.sqlite3 (backend)	Authoritative ledger (WAL mode, on the Fly volume)

10. Chat commands & shop catalog

Command	Who	What
/coins	anyone	Show your balance and owned perks
/donate	anyone	Mint a claim code, DM you the donation URL
/shop	anyone	List SKUs with prices and ownership
/buy <sku>	anyone	Purchase a SKU
/gift <player> <amount>	anyone	Transfer Valcoins to another player
/title <text clear>	perk	Set chat title prefix (chat_title perk)
/sethome	perk	Save current position (sethome perk)
/home	perk	Teleport to saved position (3-min cooldown)
/shout <message>	perk	Server-wide broadcast (consumes 1 charge)
/topdonors	anyone	Show lifetime top-5 donor leaderboard
/givecoins <player> <amount>	admin	Grant coins manually
/removecoins <player> <amount>	admin	Subtract coins manually

Shop catalog (YAML) example

```
shop:
  donor_badge:
    name: "Donor Badge"
    description: "A star next to your name in chat. Forever."
    price: 500
    effect: grant_perk          # grant_perk | add_charges
    perk: donor_badge          # internal perk id
  shout:
    name: "Server Shout"
    price: 200
    effect: add_charges
    perk: shout
    charges: 1                  # uses per purchase
```

Built-in perk handlers: **donor_badge**, **chat_title**, **sethome**, **shout**. Adding a new effect type requires editing `ShopHandler.cs::ApplyEffect`; new SKUs that reuse an existing handler are pure YAML.

11. In-game UI panel (Phase 5)

When the plugin is installed client-side, press **F8** (configurable) to open a minimal ImGui panel.

- Donate tab: one button. Calls the backend, displays your code + portal URL.
- Shop tab: scrollable SKU list with Buy buttons; shows owned/charges per row.
- Gift tab: recipient + amount fields, Send Gift button. Surfaces /shout and /title editors when you own those perks.
- Top tab: leaderboard of lifetime donors (uses /api/leaderboard/top).
- Auto-closes when you open inventory, map, or pause menu.
- Sends commands via a silent vc_action RPC so nothing leaks into public chat.

12. Verifying the full flow

- Run /donate in-game. The plugin should DM you a portal URL with an AB12-CD34 code.
- Open the URL. The portal should show your code and four provider buttons.
- Make a test donation through one provider (Ko-fi sandbox or PayPal sandbox is easiest).
- Watch backend logs for the webhook hit and a 'grants' row being written.
- Within ~10 s (your poll_interval_seconds), the plugin should pop a HUD message with the credited amount, and /coins should reflect the new balance.
- Try /buy and /gift to exercise the atomic /api/spend and /api/transfer endpoints. Retries are safe — both are idempotency-keyed.

13. Operations & troubleshooting

- SQLite + WAL on a 1 GB Fly volume comfortably handles thousands of donations.
- All webhooks are idempotent at the DB level via donations(provider, provider_txn_id) UNIQUE. Replays are safe.
- The plugin caches the last 5000 applied grant ids locally, so a crash-then-replay won't double-credit.
- OAuth states have a 10-minute TTL and are GC'd opportunistically.
- Donations below MIN_GRANT_COINS are stored with status 'rejected' for audit but are not credited.
- If a donation arrives without a recognisable claim code (donor forgot to paste it), it lands in the unmatched list. Use /api/admin/unmatched and /api/admin/credit-unmatched to reconcile manually.
- Currency conversion is JSON-configurable via COINS_PER_UNIT (defaults: \$1 = 50 coins, P1 = 1 coin).
- The plugin's local balance cache is NOT authoritative — it only answers /coins without a round-trip. The backend SQLite is the source of truth.

Common errors

- 503 from a /webhooks/* route -> the matching provider's env vars are unset on the backend. Set them and redeploy.
- Plugin logs 'Backend ready: False' -> backend_url or plugin_token in valcoin_config.json is wrong, or the backend isn't reachable from the server box.
- Plugin builds but won't load -> a Unity DLL is missing from libs/; check Section 7's table.
- Donor paid but no in-game credit -> check unmatched list first; if the donation isn't there at all, the webhook didn't fire (provider dashboard usually has a retry button).

14. Keeping this guide up to date

This PDF is produced by scripts/generate_setup_guide.py. Whenever you change one of these files, regenerate it:

- backend/app/main.py (version, routers)
- valheim-plugin/Plugin.cs (plugin version)
- backend/.env.example (env vars)
- valheim-plugin/ValheimDonationSystem.csproj (required DLLs)
- backend/fly.toml (deploy region / VM)
- backend/README.md or valheim-plugin/README.md (workflow text)
- backend/app/config.py (settings defaults)

```
# regenerate
python scripts/generate_setup_guide.py

# verify nothing drifted (CI / pre-commit)
python scripts/generate_setup_guide.py --check
```

Suggested git pre-commit hook (PowerShell):

```
# .git\hooks\pre-commit  
python scripts/generate_setup_guide.py  
git add docs/SETUP_GUIDE.pdf
```

Inputs fingerprint for this build: d334d3ae6a84. If you re-run the generator and the fingerprint changes, something the guide covers has shifted.